CSC 436 Assignment

Database Views & Normalization

Overview

In this assignment, you will enhance your group's database application by implementing SQL views for controlled data access and applying normalization techniques to achieve Third Normal Form (3NF). You will create views to restrict access to specific data and write SQL commands for managing privileges. Additionally, you will analyze your database for normalization violations, make necessary modifications to improve data integrity, and document your work with SQL queries, screenshots, and a detailed explanation of your normalization process.

Part I: SQL Views & Authorization

In this part of the assignment, you will create views in MySQL to control data access and enhance security. Views allow you to present a subset of your data to specific users without granting them full access to the underlying tables. This is particularly useful for protecting sensitive information while still providing necessary data access to different types of users.

Since the MySQL version in your cPanel does not support role-based privileges or granting user-specific permissions through phpMyAdmin, you will also write (but not execute) SQL commands that would typically be used to manage access.

A. Setup

Before creating views, complete the following:

- ☑ ~~Complete the Logging Into cPanel section to access your cPanel account.~~
- ☑ ~~Since you will be creating views, you need to verify that your database user has the necessary privileges. Complete the Updating A User's Privileges section and ensure your database user has been granted **All Privileges** in phpMyAdmin.~~
- ☑ ~~Complete the Accessing The Database section to access the database you created.~~
- ☑ ~~Review the SQL Statements section to familiarize with common SQL statements to effectively communicate with your database.~~
- ☑ ~~Review the Querying The Database section to write queries and ask questions about the data.~~

B. Creating Views

You will create at least two views that restrict access to certain data. Consider the following:

- A **limited-access view** that hides sensitive information (i.e., removing customer payment details).
- A **role-based view** that only displays records relevant to a specific user type (i.e., showing only active rental transactions for employees).

After creating your views, they will appear under the **Views** section at the bottom of your database.

C. Simulating Role-Based Access (Written SQL Only, No Execution)

Since the MySQL version in cPanel does not support granting privileges or creating roles, you will write (but not execute) the SQL commands that would typically be used to:

- Grant privileges on a view to a specific user.
- Create a role and assign users to it.

Even though these commands will not execute in your current setup, understanding them is essential for real-world database management.

Grant read-only access to a limited-access view

```
GRANT SELECT

ON Account_Public

TO 'public_user'@'localhost';

-- Create a role for employees

CREATE ROLE 'employee_role';

-- Allow employees to view active account data

GRANT SELECT

ON Account_Employee

TO 'employee_role';
```

```
-- Assign the employee role to a user

GRANT 'employee_role'

TO 'employee_user'@'localhost';
```

Part II: Database Normalization

In this part of the assignment, you will analyze and refine your group's database application design to ensure compliance with third normal form (3NF) by identifying and addressing normalization issues.

A. Database Normalization

Normalization helps eliminate redundancy, inconsistencies, and update anomalies by organizing data efficiently. You will review your current database design and apply normalization techniques as needed.

Familiarize yourself with the following rules to analyze your database for normalization violations:

- **First Normal Form (1NF)**
  - Using row order to convey information is not permitted.
  - Mixing data types within the same column is not permitted.
  - Having a table without a primary key is not permitted.
  - Storing repeating groups or multi-valued attributes is not permitted.

- **Second Normal Form (2NF)**
  - Each non-key attribute must depend on the entire primary key.

- **Third Normal Form (3NF)**
  - Every non-key attribute in a table should depend on the key, the whole key and nothing but the key.

Identify non-conforming elements in your existing database schema. Modify tables by restructuring attributes and relationships as needed to ensure that your final design meets 3NF requirements while maintaining functionality. This may involve breaking down tables, creating new tables, and redefining relationships to eliminate redundancy while ensuring data integrity.

Part III: Documentation

You will compile and explain your SQL queries, screenshots, written privilege management commands, and normalization analysis.

Document the exact SQL queries you used to define each view. Include a brief explanation of each view's purpose and expected results.

For the creation of Views, the following Querys are executed.

View 1: Limited-Access View

SQL Query

```
CREATE VIEW Account_Public AS
SELECT
  Account_ID,
  User_ID,
  Bank_name,
  Account_type
FROM Account;
```

This view restricts access to sensitive financial information by excluding the Balance column. The Expected Results would be that querying this view returns account metadata only, without exposing account balances.

View 2: Role-Based View

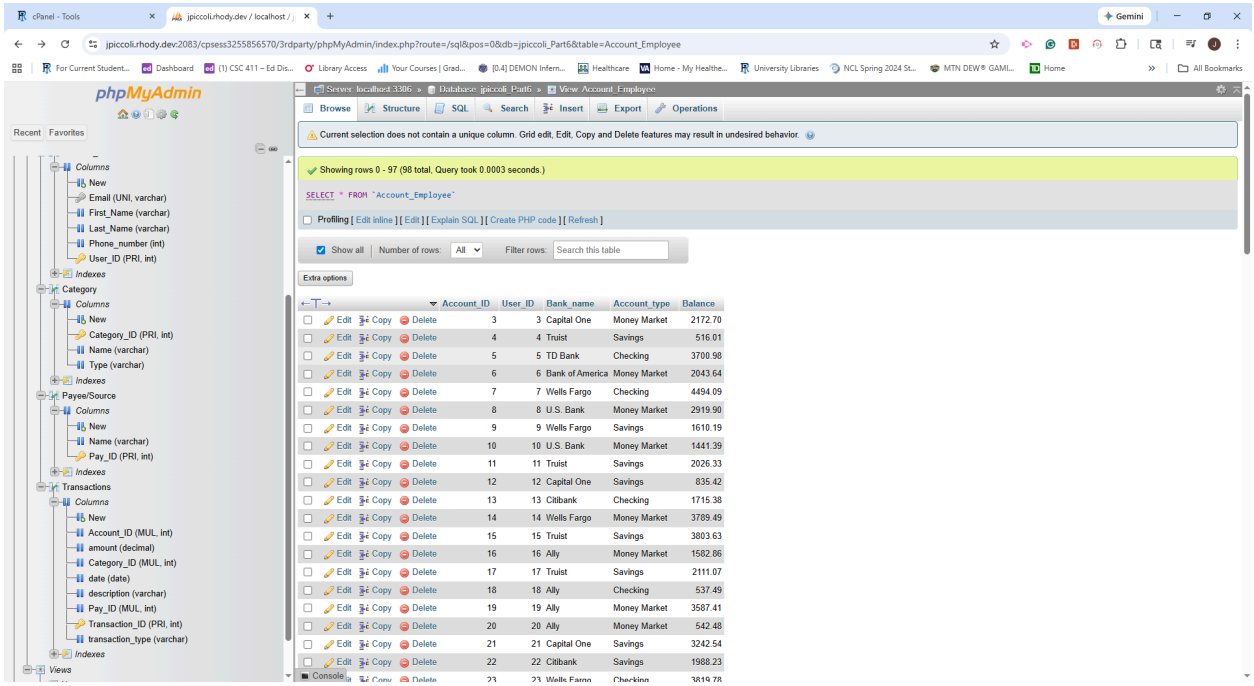
SQL Query

```
CREATE VIEW Account_Employee AS
SELECT
  Account_ID,
  User_ID,
  Bank_name,
  Account_type,
  Balance
FROM Account
WHERE Balance > 0;
```

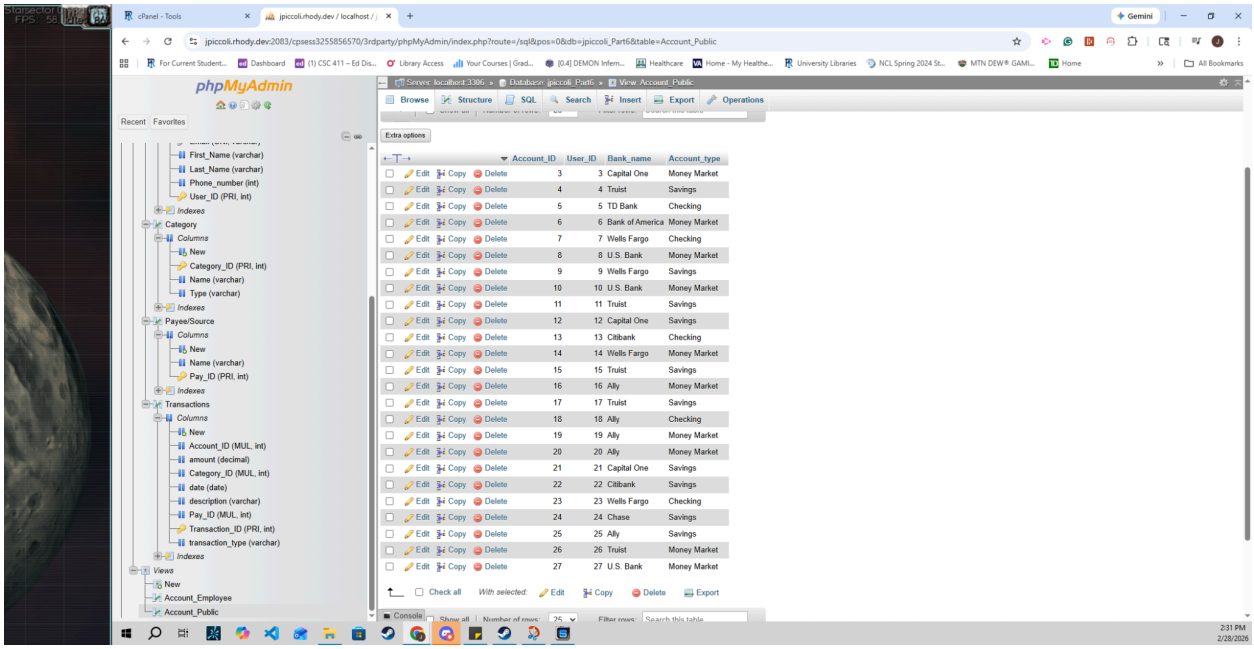
This view simulates role-based access by displaying only active accounts with a positive balance. The Expected Results are as follows: Only accounts meeting the filter condition (Balance > 0) are returned.

Take screenshots showing the data displayed under each view. You can use the Browse tab and take a screenshot of the data that is displayed under the Views section. This will demonstrate that your views were successfully created.

Under Account_Employee. We are able to see Account_ID, User_ID, Bank_name, Account_type, and Balance.



Add, while under Account_Public, we are not allowed to see the balance amount.



Include the SQL statements you would have executed to grant privileges on a view and create a role. Document each query's intended purpose, expected results, and actual SQL command.

3.

Grant access to a view
GRANT SELECT
ON Account_Public
TO 'public_user'@'localhost';

Purpose:
Grants read-only access to a limited-access view.

Expected Result:
The user can query the view but cannot access the underlying table.

Actual Result:
Not executed due to environmental limitations.

Create a role
CREATE ROLE 'employee_role'; < — fix this.

Purpose:
Defines a role to group employee permissions.

Grant view access to role
GRANT SELECT
ON Account_Employee
TO 'employee_role';

Assign role to user
GRANT 'employee_role'
TO 'employee_user'@'localhost';

Identify and explain each non-conforming element in your database schema. Describe how it violates normalization principles and why it needs to be addressed.

During schema review, all tables were evaluated for normalization violations. No repeating groups, partial dependencies, or transitive dependencies were identified. Descriptive attributes were separated into lookup tables such as Category and Payee/Source, and transactional data references these tables using foreign keys. As a result, no non-conforming elements were found.

Provide a detailed explanation of changes made during the normalization process. Justify each modification, explaining how it improves database efficiency, eliminates redundancy and inconsistencies, and ensures 3NF compliance.

The normalization process involved separating user information, account data, and transactional details into distinct tables. This eliminated redundancy by preventing repeated storage of user and category information across multiple records. Foreign key relationships were introduced to maintain referential integrity, improve query efficiency, and ensure the database complies with third normal form (3NF).

Submission

When you're finished, complete the following steps to submit your work:

- ☑ ~~Export your document with responses as a **PDF file AND save** it **inside** your **documentation** folder. Refer to the following for documentation on how to do this:~~
  - Google Docs (*File → Download → PDF Document*)
  - Microsoft Word (*File → Save As / Export → PDF*)
  - Pages (*File → Export To → PDF*)
- ☑ ~~Inside your repo, export your updated database as an **SQL file** into the **db** folder. Follow the steps in the Exporting A Database section to export your database into a single SQL file.~~
- ☑ ~~All group members should then be able to pull this SQL file from the repo and upload it to their own cPanel accounts (see Importing A Database section).~~
- ☑ ~~Upload all your changes to GitHub.~~
  - ☑ ~~**If you're using GitHub Desktop (GUI)**, complete the Uploading Changes (GitHub Desktop) section to upload your changes from your local device to GitHub.~~
  - ☑ ~~**If you're using Git (CLI)**, complete the Uploading Changes (GitHub CLI) section to upload your changes from your local device to GitHub.~~

ONE group member must paste the URL of your GitHub repository in the provided textbox in Brightspace. Click the blue *Submit* button to successfully submit your work for this assignment.

Grading Rubric

You can refer to the **Database Views & Normalization grading rubric** given in Brightspace for this assignment to find details on how your submission will be graded.